

严格无损结构化 压缩全景研究

历史方案复核 · 大结构创新 · WorldPack 路线

审计结论

OpenZL、Blosc2、跨照片方案过去只做了部分筛选，不能称整体失败。

当前首要实验

全量 descriptor similarity forest；随后补跑 ACE / BYTEDELTA。

目录

报告先复核已有证据，再从照片、数据库、描述子、匹配图和整作品对象图推导下一代结构。

执行摘要	3
零、2026-08-02 复核更正：以前并没有把所有方案用好	4
一、研究边界：什么才算 PocketWorld 的“无损”	6
二、冻结基线与本地事实	7
三、纵向研究：照片的严格无损上限与可用路线	9
四、纵向研究：SQLite、描述子和匹配数据	10
五、纵向研究：点云与 PLY	12
六、横向比较：谁解决什么问题	13
七、PWA2：建议的未来项目数据结构	14
八、当前具体实施哪里对、哪里必须改	16
九、下一代大结构：PocketWorld Lossless WorldPack	17
十、接下来的实验顺序与现实倍率	20
十一、开源商用初审	22
十二、证据台账与置信度	23
最终决策	24
主要来源	25

研究时间：2026-08-01，复核更新：2026-08-02；所属领域：移动端三维重建、机器人/世界模型数据、严格无损冷归档；研究对象类型：工程决策与开源商用可行性

执行摘要

这轮研究的结论不是再换一个“更强的万能压缩器”，而是把 PocketWorld 的未来项目从“先写成 SQLite/PLY/JPEG 文件，再把整份文件交给压缩器”升级成“先按数据语义组织成可逆的逻辑流，再由多种成熟无损编码器逐块竞争”。当前方向里的 ZPAQ 没有错，错的是让它面对混杂的 SQLite 物理页面：描述子、根节点、轨迹残差、关键点浮点数、匹配编号、页面头和 B-tree 元数据交错在一起，压缩器看不到真正的统计关系。

对当前冻结样本，SQLite 原文件为 198,983,680 B。`descriptors` 的有效负载约 160.16 MB，占 80.5%；`keypoints` 约 30.03 MB，占 15.1%；`matches` 与 `two_view_geometries` 合计约 7.13 MB。页面、索引、空闲空间不是主要矛盾。当前最优 `track_delta_v1 + ZPAQ 7.15 method 5` 为 124,401,918 B；新试的 `exact_transform_v2 + ZPAQ` 为 124,466,073 B，反而大 64,155 B，已被正确淘汰。失败原因不是“无损做不到”，而是 v2 继续在原 SQLite 页面内部对较小的列做通用转置/差分，没有把 160 MB 描述子拆成轨迹根、可预测残差和未匹配字面量，也没有把 `[N,128]` 的每个维度分成同质流。

推荐的新架构暂称 PocketWorld Lossless Archive v2 (PWA2)：

1. 照片继续独立保存为可逐字节恢复原 JPEG 的归档块；短期仍以当前已验证的 JPEG XL 精确 JPEG 转码为生产基线。跨照片方案另设研究线，不与数据库改造混在一起。
2. 将关键点与描述子作为不可拆散的逻辑记录，未来项目采用规范化顺序，并同步重映射所有匹配索引；所有 `uint8`、`float32/64` 位模式、数量和逻辑顺序可完全恢复。
3. 将描述子拆为“轨迹根”“沿 SfM 轨迹预测后的残差”“未匹配描述子”三类流；每类再按 128 个维度转成同质字节流。当前实现把这些类别混写回原来的 SQLite 行位置，这是它只额外节省 2.455% 的根本上限。
4. 匹配编号、图结构、图像 ID 和偏移量采用排序、delta、zigzag、bit-packing；关键点/位姿保持 IEEE 位模式，用 ALP/Pcodec 一类严格可逆数值编码候选；描述子流用 OpenZL、Pcodec、C-Blosc2 与 ZPAQ 做逐块选择。
5. 容器采用 MCAP 一类追加写、分块、索引、校验、可中断恢复的工程原则；最终只保留每块最小且验证通过的一个候选，不永久保留两份。
6. SQLite 对未来项目降为“管线兼容缓存”，数据真源是 PWA2。若当前原生 COLMAP 仍只能读 SQLite，则第一阶段只在完整生产管线结束后归档；只有原生管线能随机读取 PWA2 或通过适配层读取时，才允许在稀疏阶段后删除 SQLite，避免把解压等待转嫁给用户。

本轮没有发现一个成熟、公开、可直接商用、在 iPhone 上已验证且能让 PocketWorld 严格无损整体达到 5×–10× 的现成库。学术界里常见的 SIFT/点云“10×”大多通过量化、降维、词袋化、坐标体素化或只保证任务精度，违反当前边界。严格无损仍然可能继续提升，但主要来自 PocketWorld 自己已拥有的跨照片注册关系、SfM 轨迹和规范化结构，而不是压缩等级。

零、2026-08-02 复核更正：以前并没有把所有方案用好

这次复核推翻了一个过早的概括：过去的结果不能解释成“世界上的方案都失败了”。严格按仓库中的程序、实验合同、结果文件和源代码逐项核对后，只有少数方案完成了足以支持结论的测试；若干最重要的方案只跑了简化筛选，另一些只在研究文档中出现，根本没有留下可复现的输出。聊天里曾出现过的“Brunsli 已替换生产”“OpenZL 已失败”“Blosc2 已失败”“DwarFS 已测”等表述，没有得到当前仓库证据支持，不能继续当作事实。

0.1 以前到底测到了什么

方案	实际完成程度	可支持的结论	不能支持的结论
JPEG XL 0.12.0 exact JPEG	完整真机参数测试	169 张从 700,159,883 B 到 572,966,080 B；25 张 effort 10 从 106,507,504 B 到 87,282,911 B；原 JPEG 逐字节恢复	不能说明跨照片预测没有更大空间
LZMA2 preset 9 与 ZPAQ 7.15 method 5	100 MiB 真实 iPhone A/B	同一输入下 ZPAQ 62,850,614 B，小于 LZMA2 的 68,045,272 B，且 SHA 一致	不能说明 ZPAQ 是所有结构化数据的终极后端
track_delta_v1 + ZPAQ	完整主机三轮	198,983,680 B DB 到 124,401,918 B；三轮确定、逐字节、SHA、integrity 全过	尚未通过真实 iPhone 生产线准入
exact_transform_v2 + ZPAQ	完整主机否决	124,466,073 B，严格无损但比 track 基线大 64,155 B	只否决这一个变换，不否决列式结构本身
PWA2 全 ZPAQ	完整主机结构实验	129,567,942 B；逻辑值、顺序、随机读、物化 SQLite integrity 一致	这个具体 PWA2 结构比 track 基线大，不代表所有新结构失败
Pcodec v1.0.2 level 12	66 个兼容数值成员全部测试	数值成员 8,612,716 B 到 8,246,273 B；PWA2 同作用域新基线 129,201,499 B	只改善 PWA2 数值成员，未成为全 DB 或生产赢家
OpenZL	仅 6 个成员、通用 numeric brute-force selector	这个通用配置在这 6 个成员上未胜 ZPAQ	没有跑官方 parser + clustering + ACE training，不能称 OpenZL 失败
C-Blosc2	仅 6 个成员的 none/shuffle/bitshuffle	这三组简单配置未胜 ZPAQ	没有测 BYTEDELTA、split 策略、B2ND chunk/block 与字典，不能称 Blosc2 失败
跨照片 sfm_coeff_group_4/8_xz9e	简化原型，16/37 张后按旧门槛提前停止	简单顺序分组、前一帧预测、稀疏投影投票这一路不够强	没有复刻论文的全局参考树、混合视差补偿、局部 DCT 块搜索和自适应频域编码，不能否决跨照片方案
Brunsli、Lepton、PackJPG、libbsc、Kanzi、DwarFS、Parquet/Arrow、ORC、TileDB	当前仓库无耐久 benchmark 输出	只能称“研究过或提议过”	不能声称优于、劣于或已经替换生产

这张表回答了用户最关心的问题：是的，以前有几条关键路线没有好好用。Pcodec 的数值后端测试相对完整；OpenZL、Blosc2 和跨照片结构只是初筛；许多名称根本没真正跑。今后报告必须用“完整测试、部分筛选、未测试”三态，不再把“没过一个配置”写成“整个项目不合格”。

0.2 三处最严重的实施偏差

第一，OpenZL 官方把专用压缩分成“解析结构”和“训练后端”两阶段，并明确推荐同质样本使用 ACE，多个结构使用 clustering + ACE；默认图不足时再接自定义 codec。旧实现注册的却是 `official_numeric_array_brute_force`，没有训练器、没有数据格式 parser，也没有用轨迹/描述子图给 ACE 种子。官方文档所说的最强路径正好被跳过了。OpenZL 使用指南与训练资源说明还说明 100 MB ACE 全训练可能需要约 3,000 秒，并建议先用少量代表文件训练；这进一步证明一次通用 6-member screening 不是完整复刻。

第二，C-Blosc2 旧桥的可逆白名单只有 `NOSHUFFLE`、`SHUFFLE`、`BITSHUFFLE`。官方 BYTEDELTA 是先按字节重要性分流再做差分，并可与 split + Zstd 组合；官方 ERA5 数值数据中，某些数据集从 4.1×提高到 5.7×，但也有数据集

变差，因此必须在 PocketWorld 真数据上系统搜索。旧测试没有 BYTEDELTA、没有 always-split/never-split、没有 [N,128] 的 B2ND 双层分块，也没有训练字典。[BYTEDELTA 官方实验](#)和 [B2ND 官方说明](#)支持的是“值得完整测试”，不是保证一定赢。

第三，旧跨照片原型把照片按拍摄顺序每 4/8 张分组，只用前一张作参考；稀疏 SfM 投影没有覆盖的 DCT 块直接退回同位置映射。实际映射覆盖率只有 12.85%/12.16%。微软 2016 方法则先在整个集合上按特征域预测代价构造伪视频/参考关系，再做全局与局部混合视差补偿，最后在频域自适应去冗余，论文报告平均节省超过 31%。旧原型只验证了一个弱启发式，不是论文复现。[微软原论文页面](#)

0.3 清理结果：删的是占空间的可再生物，不删失败证据

本轮已删除确认可再生的旧 codec 源码缓存、构建目录、600 MB Python 虚拟环境、旧目标文件和压缩 benchmark 临时日志，累计释放约 2.21 GB。目前 [experiments/](#) 只有约 1.7 MB，[.context/compound-engineering/ce-optimize/](#) 约 1.6 MB，保存的是实验合同、输入哈希、结果 JSON/YAML、DVC/MLflow 元数据和极小的复现源码。

这些约 3.3 MB 的证据不继续删除：它们证明哪些失败是真的、哪些只是没测完整，也符合“失败实验必须保留”的可复现要求。删除它们几乎不省空间，却会让下一次又重复跑几十轮。与压缩无关、名称显示属于 portable-SfM/其他共享工作的临时目录没有动，避免误删另一条正在进行的开发。

一、研究边界：什么才算 PocketWorld 的“无损”

1.1 三层无损边界必须分开

历史文件归档要求最严格：解压后的原 JPEG、旧 SQLite、旧 PLY 必须与输入逐字节一致，SHA-256 完全相同。任何改变 SQLite 页面排列、JPEG marker 顺序、PLY 属性顺序的方案，即使数值一样，也不能冒充旧文件精确恢复。

未来规范化格式可以采用“数据内容严格无损”：

- 128 维描述子的每个 `uint8` 完全一致；
- 关键点、位姿、相机参数、点云的每个 IEEE `float32/float64` 位模式完全一致，包括负零、NaN 载荷等特殊值；
- 匹配编号、数量、方向与逻辑顺序一致；
- 每个块和整份逻辑流都有 SHA-256；
- 照片仍能恢复成原 JPEG 文件并通过整文件 SHA-256；
- 如临时物化成 COLMAP SQLite，查询到的表、BLOB 和关联完全一致，但不保留没有训练价值的 B-tree 页面布局、空闲字节和物理写入历史。

这不会丢掉机器人训练、世界模型、重建复现所需的数据。失去的只是数据库“当初先写哪一页、哪一页曾被删除”的物理痕迹；它不是样本内容，也不参与模型训练。对旧项目仍保留整文件精确恢复，对未来项目则把规范化归档本身定义为原文件。

1.2 不允许偷换为“任务无损”

大量论文把“检索准确率基本不变”“点云几何看起来一样”“PSNR 很高”称为 near-lossless、visually lossless 或 performance-preserving。这些都不满足本项目。下列行为全部排除：

- SIFT/PQ、OPQ、产品量化、PCA、二值化、词袋化；
- `float32` 转 `float16`、截断尾数、允许误差的 `zfp/fpzip` 模式；
- Draco/G-PCC 的坐标量化或把浮点点云先体素化为整数网格；
- 解码后重新编码 JPEG，只保证像素相同却不能恢复原 JPEG bitstream；
- 删除可重算的特征、匹配、中间点云；用户已经明确要求全部保留。

1.3 生产约束

归档是冷数据任务，压缩可以慢，但不得延长用户正在拍摄或训练的等待。生产管线运行时必须暂停归档；App 活跃且管线空闲时可直接运行，系统空闲时由 `BGProcessingTask` 继续。系统中断只能发生在块事务边界；恢复时不能重做已提交块。删除原件的条件是：候选完成、逆变换完成、逐字节/逐流 SHA 验证通过、索引和审计记录原子提交。最终静态存储只留一份获胜归档，临时双候选只能存在于受控事务中。

二、冻结基线与本地事实

2.1 可复现实验身份

- PocketWorld Git HEAD : 13d2a4f05d491464c537a9135496eccaa05c2358
- 输入项目 : cap_1785512421333592
- SQLite 大小 : 198,983,680 B
- SQLite SHA-256 : 0c12c0dfa76d50cae59929774242282d8236daeb852bdee6ac99c3062d6d08b0
- SQLite PRAGMA integrity_check : ok
- ZPAQ : 7.15 , method 5 , 冻结二进制 SHA-256 为 e85ec2529eb0ba22ceaeabd461e55357ef099b80f61c14f377b429ea3d49d418
- 证据角色 : 主机预筛 , 不是生产真机准入 ; 生产赢家仍必须在真实 iPhone、真实移动管线复验。

2.2 SQLite 组成

表/对象	分配字节	payload 字节	约占原 DB
descriptors	160,378,880	160,161,491	80.5%
keypoints	30,121,984	30,031,454	15.1%
matches	4,435,968	3,785,140	1.9%
two_view_geometries	3,969,024	3,340,645	1.7%
其余 schema/索引/小表	< 0.1 MB	< 0.1 MB	可忽略

这张表直接否定了“先 VACUUM、换 WITHOUT ROWID、压索引就能再省很多”的假设。SQLite 页面利用率已经很高，结构性收益必须来自 160 MB 描述子本身。

2.3 已有结果

方案	归档大小	相对原 DB	相对 raw ZPAQ	严格恢复
raw SQLite + ZPAQ	127,533,074 B	1.560×	基线	通过
track_delta_v1 + ZPAQ	124,401,918 B	1.599×	小 2.455%	三轮 SHA/逐字节/integrity 通过
exact_transform_v2 + ZPAQ	124,466,073 B	1.599×	比 track 大 64,155 B	通过，但尺寸门槛失败

track_delta_v1 从 386,890 条已验证匹配边中形成确定性生成森林，覆盖 306,740 个匹配描述子节点，只预测 224,119 个节点。总描述子约 1,251,246 个，所以真正使用轨迹预测的只有 17.9%，另有 82.1% 仍作为原字节交给 ZPAQ。即使已预测的残差也被写回原物理行位置，与根和未匹配字节面量交错。

exact_transform_v2 对约 30 MB 关键点做浮点字节平面/XOR，对约 7 MB 匹配数据做列转置/delta。它保持了可逆性，却没有改变主导的描述子数据模型，且通用变换轻微干扰了 ZPAQ 上下文，因此比 track_delta_v1 大 0.052%。这个失败模式非常有信息量：下一轮不能继续在 SQLite 页内叠加小变换。

2.4 PWA2 与专用后端的最新结果

PWA2 把 SQLite 逻辑内容拆成 109 个成员，保留了 1,251,246 个描述子节点、1,251,246 条关键点记录、470,954 条 match 记录和 386,890 条 two-view 记录。它在逻辑 SHA、所有 cell、行数/顺序、随机读取和重新物化 SQLite integrity 上全部一致，但全 ZPAQ 容器为 129,567,942 B，比 `track_delta_v1` 大 5,166,024 B。主要原因不是 ZPAQ 变差，而是结构仍只有 224,119 个 predicted descriptors，另有 82,621 个 roots 和 944,506 个 unmatched literals；分流/索引边信息增加了，预测覆盖却没有扩大。

Pcodec v1.0.2 level 12 随后完整覆盖 66 个兼容数值成员，66 个都比同成员 ZPAQ 小：8,612,716 B 降至 8,246,273 B，净省 366,443 B。由此 PWA2 研究作用域的新基线是 129,201,499 B。它确实是局部进步，按照“严格可比、严格无损、哪怕只小 1 B 也升级同作用域基线”的新规则应被接受；但它仍大于生产/全 DB 的 `track_delta_v1` 124,401,918 B，所以不能混称为全局生产新基线。

旧报告中的 111,961,726 B“至少再小 10%”门槛已经被用户撤销。今后不再用 10% 一刀切淘汰局部进步；同时，大结构研究仍按“对完整作品贡献”排序，避免为了几百 KB 长期侵入高风险生产层。

三、纵向研究：照片的严格无损上限与可用路线

3.1 成熟生产基线

JPEG XL 官方站点与 libjxl 官方仓库明确支持对传统 JPEG 做可逆转码并恢复原 JPEG bitstream。它有成熟实现、开放规范和移动端工程基础，因此当前已经验证过的 JXL 仍是稳健生产基线。

Brunslis 与 Lepton 也属于“恢复原 JPEG 文件”的专用重压缩，而不是把像素重新编码。它们可以作为逐文件候选，但没有必要让所有项目永久保存三份。正确做法是临时生成候选、立即逆解和 SHA 验证，只提交最小者。Brunslis 官方 C 编码 API 以完整输入、单次编码为单位，因此不支持在一张 JPEG 的中间保存内部编码状态；但这不妨碍项目级断点续传：每张照片是一个事务块，管线开始时不再领取下一张，在途一张完成或取消后记录状态即可。

3.2 跨照片冗余是照片继续提升的唯一高价值方向

微软研究院与天津大学的 [Lossless Compression of JPEG Coded Photo Collections](#) (IEEE TIP 2016, DOI 10.1109/TIP.2016.2551366) 不是简单把 JPEG 拼包，而是先按视觉特征的预测代价构造照片树，再用视差补偿、空间预测和 DCT 域残差编码恢复每张原 JPEG。论文报告照片集合平均比原 JPEG 总量减少超过 31%。SfM 版本 [Incremental SfM based lossless compression of JPEG coded photo album](#) 进一步利用相机位姿和稀疏点云做投影、三角化和 warping；这与 PocketWorld 已经拥有的输入高度吻合。

后续还有 LLJPEG 的 DCT 域 intra prediction，以及 CVPR 2022 的 [Practical Learned Lossless JPEG Recompression](#)，后者按 JPEG 频率重排 DCT 系数并用跨色彩通道熵模型。研究结果普遍支持同一个判断：要恢复原 JPEG，应该预测原 JPEG 的量化 DCT 系数、表和 marker 信息，而不是先解成 RGB，再用普通无损图像编码。

但这些研究不能直接批准生产：

- 没找到 2016 集合方案或 LLJPEG 的成熟公开实现；论文不是软件许可证。
- SfM 论文中的 HEVC-like/CABAC 组件带来实现与潜在专利审查负担。
- 论文数据集上的 26%–35% 减少不能直接外推到 PocketWorld 连续 12MP 照片。
- 旧实验曾预登记 $2.165\times$ (减少约 53.8%) 作为停止门槛，公开论文没有证明达到该倍率；用户后来取消了把它作为所有局部方案的硬淘汰线，因此它只保留为历史实验条件。
- 学习式方案通常依赖 GPU/模型，论文也没有 iPhone A16 上的随机 4/8 张读取、峰值内存和生产总时长证据。

因此照片研究的正确下一步是做一次忠实结构 A/B，而不是继续修补旧原型：在约 100 MB 连续照片上构造全局参考树，加入真实位姿/SfM 约束、局部频域块搜索和完整边信息，再与当前 JXL 精确基线比较。任何严格更小且解码边界合格的结果都成为研究基线；是否进入生产再按完整作品净收益、实现成熟度、许可和真机资源综合决定。

四、纵向研究：SQLite、描述子和匹配数据

4.1 SQLite 容器不是未来冷归档真源

SQLite 文件格式的目标是事务、随机页更新和兼容查询，不是表达计算机视觉数组之间的统计关系。[VACUUM](#)可以回收空闲页并重排页面；[WITHOUT ROWID](#)可避免某些主键索引重复，但对本样本的大 BLOB 表没有 10% 级空间可回收。SQLite 官方 ZIPVFS 还是商业扩展，而且它仍只压页面，不理解描述子轨迹。

[phiresky/sqlite-zstd](#) 证明了“按列/行透明压缩 SQLite”在高重复 JSON 上可以非常有效，但项目 README 本身称实现实验性，不建议关键数据依赖；其 LGPL-3.0 也需要单独评估移动 App 链接和分发义务。更重要的是，JSON 示例不代表高熵的 SIFT [uint8\[128\]](#)。

因此未来架构不应问“怎样把 SQLite 再压一点”，而应问“怎样让描述子、关键点、匹配、轨迹成为一等逻辑流”。SQLite 只在需要 COLMAP 兼容时物化。

4.2 为什么文献中的描述子 5×–16×常常不适用

[Survey of SIFT Compression Schemes](#)汇总的大倍率路线主要使用降维、量化、向量量化、哈希或容忍检索性能损失。移动视觉搜索论文中的“低 bit rate”通常优化检索率，不恢复每个 128 维字节。把这些数字直接当作严格无损承诺，会混淆两种问题。

严格无损仍有三类真实机会：

1. 去掉任意顺序的信息。集合/多集合编码可以不为无意义排列付费。[Compressing Sets and Multisets of Sequences, Toward a source coding theory for sets](#)和 2024 年的 [Practical Shuffle Coding](#)都说明这在理论上可逆。但 PocketWorld 的描述子与关键点、匹配索引绑定，不能单独排序；未来规范必须共同排序并重映射引用。按本样本每图约 8,874 个描述子估算，纯 $\log_2(n!)$ 顺序信息仅约 1.83 MB，约占描述子 1.14%，不可能单独带来 5×。
2. 利用同一 3D 轨迹上的相关性。同一个实体点被多张连续照片观察，对应描述子比任意两行更相关。当前 track delta 已证明方向有效，但生成树按稳定 ID 选边，不按残差代价优化，而且只覆盖一部分节点。
3. 把不同统计群体分开建模。轨迹根、轨迹残差、未匹配特征、128 个维度、关键点列、匹配 ID 的分布不同。先分流，再做 byte-lane、delta、bit-packing 和熵编码，才可能超过 ZPAQ。

4.3 OpenZL：方向最吻合，但不能直接上生产

Meta 的 [OpenZL](#)把结构化输入先 Parse/Group 为同质流，再把 delta、FieldLZ、transpose 和熵编码组成可逆图；一个通用解码器读取帧内记录的图。这与 PWA2 所需的“轨迹/列/残差分流”最吻合。Meta 的[官方介绍](#)也明确把 array-of-structs 转为 structure-of-arrays 视为关键步骤。

但 OpenZL v0.2.0 仍是候选框架，不是现成描述子编码器：

- 官方限制文档说明目前没有真正流式接口，大 payload 内存开销高，超过约 500 MB 的行为未定义；手机必须分成小块。
- 128 维描述子是二维张量且有跨轨迹关系，通用自动训练不会天然知道几何图；仍需 PocketWorld 显式分流或自定义 parser/graph。
- 官方没有给出 iOS/A16 的支持与资源证据。
- GitHub issue #116 中，社区用户对结构化 protobuf 训练曾得到比 Zstd-19 更小的结果，但同时遇到训练破坏和解码接口误用，维护者承认近期改动破坏了训练。这是“结构感知有效、集成成熟度不足”的典型证据，不能当生产稳定性证明。

商用审查暂定 conditional：固定 **v0.2.0** (tag revision **3dceb64867840201fb8f57a29d179995f700c9b8**)，核心为 BSD 风格许可证，直接依赖的 Zstd/LZ4 许可方向友好；但进入 App 前仍需完整 transitive/NOTICE/patent/iOS 构建审计。不得跟随 **main** 或动态训练结果进入永久格式。

4.4 Pcodec/Pco：数值流的强候选

Pcodec是 Apache-2.0 的 Rust 数值压缩库，支持无损整数和 IEEE 浮点数组，采用模式、delta、分桶与 tANS，并提供 chunk/page/batch 分层和页面级随机解码。其论文 [Pcodec: Better Compression for Numerical Sequences](#)在六类数值数据上报告比比较对象更高的压缩率及高解码吞吐，但这不是 PocketWorld 数据证据。

它适合图像 ID、行号、匹配编号、轨迹父节点、偏移量和拆列后的关键点。对描述子不能把全部 160 MB 当作一条 **u8** 序列；官方也提醒混合语义序列和内在二维数据可能效果差。应先形成 128 个维度流或轨迹残差流。固定候选为 **v1.0.2**，许可证结论暂为 conditional：核心 Apache-2.0，但官方 C bindings 不完整，iOS 需要自有极薄 Rust **staticlib** ABI、锁定 Cargo 依赖并真机验证。

4.5 ALP、Parquet、Arrow、Blosc2、TileDB 的正确角色

ALP (SIGMOD 2024, DOI [10.1145/3626717](#)) 对 IEEE 浮点进行可逆 decimal factoring，失败值走 ALP-RD 位拆分；其 artifact 获得可用、可复用与结果复现徽章，也在 Apple M1 ARM64/NEON 上有复现实验。它非常适合 30 MB 关键点/位姿候选，但对主导的 **uint8[128]** 描述子帮助有限。MIT 许可方向友好，仍缺 iPhone 证据。

Parquet 编码规范提供 **DELTA_BINARY_PACKED**、**DELTA_BYTE_ARRAY** 和 **BYTE_STREAM_SPLIT**。规范明确说明 **BYTE_STREAM_SPLIT** 本身不减小字节，只把每个值相同字节位置放入同一流，供后续压缩器利用；这证明 **exact_transform_v2** 的 byte-plane 思想没有错，只是它用在了小表，并且仍嵌在 SQLite 页面中。Parquet/Arrow 适合作为结构与互操作参考，但完整 Arrow/Parquet 依赖对 iPhone 和一个 200 MB 项目过重，也没有轨迹预测器。

C-Blosc2/B2ND提供 C99、分块、多维切片、partial chunk read、Zstd 字典、delta/shuffle/bitshuffle 管线和 ARM NEON shuffle。它的 BSD 许可和 C ABI 比 Arrow 更适合移动端。严格无损时必须禁用 **trunc_prec**、**INT_TRUNC**、**NDMEAN**、有损 **zfp** 等过滤器。它可作为 **[N,128]** 分块和局部读取实现，也可测试 **BYTEDELTA/SHUFFLE/BITSHUFFLE + Zstd**；但它仍不知道 SfM 轨迹，必须在 PocketWorld 预处理之后。商用结论为 conditional，需固定版本并审计构建时可选 bundled codecs。

TileDB把每个 attribute、坐标和 offset 分文件，tile 是 I/O 与压缩原子；官方也强调 tile 形状和数据语义决定实际压缩率。它验证了“列式、按语义分流、分块随机读”的架构共识，但作为手机内嵌存储引擎过重，且没有描述子轨迹专用编码；不建议作为第一生产实现。

4.6 Shuffle Coding 的位置

无序集合熵编码是值得保留的研究臂，尤其适合不要求原顺序的匹配边集合。当前 [juliuskunze/shuffle-coding](#) 是 Rust 研究实现，仓库 [Cargo.toml](#) 声称 MIT，但固定 revision 未发现独立 LICENSE 文件，且没有 iOS 证据，所以商用状态为 insufficient-evidence。它不应挡住主路径；等描述子覆盖与结构分流稳定后，再评估它能否对图结构继续产生严格净节省。

五、纵向研究：点云与 PLY

5.1 G-PCC/Draco 的“大倍率”为什么不是当前答案

MPEG G-PCC/TMC13 常见“lossless 最高 10:1”指的是体素化、整数坐标的几何语义内无损。原始浮点坐标通常先缩放、平移、取整到整数网格，这一步已经不能恢复任意 PLY 的原 float 位模式、记录顺序、属性顺序和文件字节。TMC13 的 COPYING 还明确提示实现可能涉及专利且没有授予专利权。因此即使代码是 BSD 风格，商业结论仍是 block/conditional pending patent license，并且语义先不合格。

Draco 是 Apache-2.0；关闭 position quantization 只能说明不执行该量化步骤，官方并未承诺恢复原 PLY 文件 bitstream 或记录顺序。它可以作为未来“规范化点云逻辑流”的实验臂，不能替代旧 PLY 精确归档。

LASzip/LAZ 对 LAS 的整数/scale-offset 记录可以精确，但 PocketWorld 是任意 PLY 浮点与属性布局，不能直接套用。大厂点云压缩之所以经常达到很高倍率，是因为它们先规定了整数网格、属性精度和允许误差；这正是当前项目不允许做的源头缩减。

5.2 严格浮点可逆候选

zfp reversible mode 明确支持逐 bit 恢复浮点，包括特殊值；fpzip 也提供精确模式与 BSD 许可。它们利用浮点指数/尾数和网格邻域，适合规范化坐标/法线/颜色列。公开 LLNL 示例也表明科学浮点的严格无损比率可能只有约 $1.04\times-1.11\times$ ，收益高度依赖数据。

由于当前 PLY 只有约 1.2–8 MB，它不是整体体积第一矛盾。推荐保持 ZPAQ 基线，等描述子架构稳定后再用 zfp/fpzip 精确模式做小测试；即使多省 30%，对整个作品贡献也很小。

六、横向比较：谁解决什么问题

方案	解决层级	严格无损	随机读/分块	iPhone 嵌入性	对 160 MB 描述子潜力	当前结论
ZPAQ method 5	通用最终熵压缩	是	弱	已有生产路径	已知基线	保留为每块回退
OpenZL v0.2.0	结构解析+可逆图+熵编码	是	当前不足	未证实	高，但需自定义流	主机优先候选
Pcodec v1.0.2	数值序列/页面	是	强	Rust 桥需验证	中高，需 128-lane	主机+真机候选
ALP	浮点列	是	分块可设计	ARM 有证据，iOS 未证实	低（不适合 u8）	keypoints/poses 候选
C-Blosc2/B2ND	多维块+可逆 filters	是，需禁用有损 filter	强	C99/ARM 较好	中，需轨迹预处理	移动端容器候选
Parquet/Arrow	列式标准/编码	是	强	依赖较重	中低，无轨迹模型	参考/互操作，不优先嵌入
TileDB	数组数据库/tiles	是	强	过重	中，无轨迹模型	架构参考
Shuffle Coding	无序集合熵	是	需自建	未证实	只省顺序/图边	后置研究臂
G-PCC/Draco	点云几何	常含量化/规范化	有	复杂	不相关	不可用来宣称旧 PLY 严格无损
zfp/fpzip exact	浮点数组	是	块级	C/C++可行	不相关	点云小规模候选

横向结论：OpenZL、Pcodec、Blosc2、Parquet、TileDB 并不是互斥的“压缩算法冠军”。它们共同证明了一条工程共识：先把结构解析成同质列/块，再对每类数据选择可逆变换和后端编码器。PocketWorld 的优势是比通用库多掌握一层 SfM 语义：哪两个描述子属于同一轨迹、哪个关键点对应哪个描述子、匹配图怎样连接。只有把这层关系送入格式，才能超过通用列式方案。

七、PWA2：建议的未来项目数据结构

7.1 容器层

借鉴 MCAP 的 append-only、chunk、index、summary、CRC 和中断后可恢复原则，而不是直接采用其 Zstd/LZ4 作为压缩赢家。每份归档由固定头、schema、多个不可变 chunk、尾部索引和审计清单组成。每个 chunk 记录：

- `schema_version`、逻辑流 ID、项目 ID、顺序范围；
- 原始逻辑字节数和 SHA-256；
- transform ID/version/config 与 codec ID/version/config；
- 压缩后大小、压缩块 SHA-256；
- 依赖块 ID，例如 residual chunk 依赖 root/parent-map chunk；
- 状态 `staging` -> `verified` -> `committed`；
- 完成时间、设备、App build marker 与管线 revision。

块大小先预登记 4、8、16 MB 三档。OpenZL 当前高内存开销意味着不能把整个 200 MB DB 一次性喂入。4–16 MB 也让 BGProcessingTask 在块边界安全暂停，并支持后续随机读取。

7.2 相机、图像与位姿流

相机 ID、图像 ID、时间戳、宽高、模型枚举和偏移量分列；单调字段 delta + Pcodec，布尔/枚举 bit-pack。位姿的四元数/平移保留原 float64 位模式，按分量分列，用 ALP、Pcodec 或 ZPAQ 逐块选小。绝不重新归一化四元数，因为那会改 bit。

7.3 关键点与描述子共同规范化

每个 keypoint-descriptor pair 是一个逻辑记录。对未来项目可定义确定性的 canonical key，例如 (`image_id`, `track_id_presence`, `track_id`, `keypoint_y_bits`, `keypoint_x_bits`, `original_ordinal`)；只要 `original_ordinal` 或完整逆置换在需要恢复旧逻辑顺序时保存，就不会丢记录。所有 match row index 按同一置换重映射。规范化排序的主要作用不是省掉 1.8 MB 顺序信息，而是让同轨迹和相似上下文相邻。

关键点拆为 `x/y/scale/orientation/...` 列。每列按原始 bit pattern 读取，候选为 ALP exact、Pcodec float、BYTE_STREAM_SPLIT+ZPAQ；逐块只保留最小者。

7.4 描述子轨迹编码

描述子分为：

1. `track_roots`：每个轨迹选一个完整 `uint8[128]` 根；
2. `track_residuals`：子描述子相对父描述子逐维 modulo-256 或 signed delta；
3. `unmatched_literals`：未进入验证轨迹的完整描述子；
4. `track_topology`：父节点、轨迹边、原 image/row 引用；
5. `permutation/remap`：若逻辑接口要求原顺序，保存可逆映射。

父节点不应只按 ID 构造稳定生成树。下一实验可在已验证匹配图内使用固定、可复现的最小残差代价森林：边代价为 128 维 residual 的预估编码 bit 数，排序键追加 ID 保证确定性。必须把 parent-map 开销计入，且防止为了选树花费不可控内存。研究时同时保留当前 DSU 森林作为 A/B 基线。

每类描述子再转置成 128 个 byte lanes。对 residual 可比较：

- modulo-256 原字节；
- signed difference + zigzag；
- 每维 OpenZL FieldLZ/delta/entropy；
- 每维 Pcodec u8/i16；
- C-Blosc2 BYTEDELTA/SHUFFLE/BITSHUFFLE + Zstd；
- ZPAQ 直接压整个同质流。

这些都是可逆候选。编码器对每个 chunk 生成候选、逆解验证，提交最小者。格式必须记录 codec 版本与参数，不能依赖“未来默认值”。

7.5 matches 与 two-view geometry

按 (image_id1,image_id2) 排序图像对；match pair 的两个 uint32 分列，按行排序后 delta/zigzag/bit-pack/Pcodec。two-view 的矩阵和姿态浮点按列保持 bit，inlier mask 单独 RLE/bit-pack。若未来 pipeline 认为匹配边集合顺序无语义，可把 canonical order 定义为新格式的原顺序；若任何下游依赖原顺序，则保存逆置换并纳入 SHA。

7.6 点云与照片

点云 header/schema、坐标、颜色、法线、置信度分别成流。旧 PLY 保持“文件包 + ZPAQ”；未来 PWA2 可对 float 列试 zfp reversible/fpzip exact，并记录逻辑 PLY exporter revision。照片继续一张一块；归档索引保存每张原 JPEG SHA、获胜 codec 和偏移，支持随机读 4/8 张。

7.7 只留一份与中断恢复

“多候选选择小”不等于永久双份：

1. 原块仍是唯一权威；在 staging 目录依次生成候选，或者同一时间只保留当前最小候选；
2. 每个候选立即解码到流式校验器，比较长度、SHA 和逐字段不变量；
3. 最小合格候选 fsync，写入索引事务；
4. 提交成功后删除原块与落选候选；
5. 系统中断时只清理未提交 staging，已提交块不回滚；
6. 项目仍在生产管线时不领取新块。

这满足最终只存一份，也避免未验证候选替换原件。

八、当前具体实施哪里对、哪里必须改

8.1 已经正确的部分

- ZPAQ 7.15 method 5 作为成熟、高比率、严格恢复基线是合理的。
- `track_delta_v1` 使用 verified matches，而不是任意相邻行；这个局部思路被实测证明有效。
- 确定性排序、正逆变换、三轮 SHA、逐字节、SQLite integrity 和不改源文件的门禁是正确的。
- 背景队列、临时文件、原件最后删除、生产管线运行时暂停的事务思想适合新容器。
- 宿主预筛失败后不浪费真机时间是正确流程；`exact_transform_v2` 应维持 rejected 证据，不上手机。

8.2 需要停止的部分

- 不再继续给同长度 SQLite 文件叠加第三、第四个通用 byte transform。
- 不再把换压器当成主创新：libbsc、Kanzi、LZMA2、ZPAQ 只会看到同一混杂字节流。
- 不再用论文中的 lossy descriptor/G-PCC headline ratio 估算严格无损产品。
- 不在生产永久格式中跟随 OpenZL main、自动训练图或未冻结模型。
- 不因“可重新生成”删除任何照片、特征、匹配、稀疏/最终点云。

8.3 应该保留但改变位置的组件

`track_delta_v1` 不应被丢弃，而应升级为 PWA2 的一个结构前端基线。ZPAQ 不应被替换，而应降为每个逻辑 chunk 的成熟 fallback。Blosc2/Pcodec/OpenZL 不应直接包 SQLite，而应只接收已拆好的同质流。

九、下一代大结构：PocketWorld Lossless WorldPack

继续把 JPEG、SQLite、PLY 当成三个孤立文件，然后在每个文件后面轮流尝试 ZPAQ、Kanzi、libbsc、LZMA，是在同一层做局部搜索。新的大结构应把一个作品视为有类型、有引用关系、有相似关系的不可变对象图。暂称 **Lossless WorldPack**，它不是新熵编码器，而是把成熟技术组合到正确层级：内容寻址、相似对象聚类、参考树/差分、语义列式流、训练熵模型、随机读索引和逐块事务。

9.1 第一层：内容寻址和跨文件/跨项目精确去重

每个逻辑对象先按语义边界切块并计算 SHA-256：原 JPEG bitstream、JPEG DCT 数据、descriptor chunk、match adjacency、pose/keypoint columns、PLY 属性块、模型/配置和审计清单。完全相同的块只存一次，文件只是有序块 ID 清单。Restic 的官方设计用 Rabin 内容定义分块处理任意位置插入，块平均约 1 MiB，并以 SHA-256 内容 ID 组织不可变对象；这证明了精确恢复、增量写入和去重可以同时成立。[restic 设计](#)

对 PocketWorld，普通 CDC 不应直接切已压缩 JPEG，因为微小像素变化会让后续 JPEG bitstream 大面积改变、去重命中低。它主要用于以下内容：重复配置、相同模型、重复导出、同一项目不同阶段复制的 DB/PLY 区段、跨项目复用的标定/元数据，以及语义变换之后形成的相同 descriptor/match/geometry chunks。云端可在同一用户或同一租户域内去重；跨租户去重存在内容确认与隐私侧信道，不应默认启用。

9.2 第二层：相似对象排序、参考树和精确 delta

真正的跃迁来自“近似相同但字节不完全相同”的对象。Git packfile 会寻找名称/大小相近的对象，保留一个完整对象，其他对象保存相对 delta；DwarFS 使用相似哈希聚类文件，使压缩器能够跨文件边界利用冗余。DwarFS 官方展示的数十倍、上百倍倍率来自大量版本/近重复文件，不是普通照片的普适承诺，但它证明先排序/聚类相似对象，再压缩可以比原目录顺序高出数量级。[Git Packfiles](#)、[DwarFS 官方仓库](#)

WorldPack 采用有向无环参考森林：每个对象只能引用编号更早的父对象；根对象完整保存，子对象保存父 ID 与精确 residual。编码端可花很长时间找最小代价父节点；解码端只按已记录的边恢复，不运行搜索。参考链长度固定上限，常用/最新对象优先作为根，避免为了压缩率把随机读取变成解几十层链。通用二进制 delta 可用 VCDIFF 作为诊断基线，但生产实现更应针对 JPEG DCT、descriptor lanes 和 graph IDs 建模；Google OpenVCDIFF 已在 2026 年归档，只适合作为 Apache-2.0 参考实现，不宜直接成为新永久格式依赖。[OpenVCDiff](#)

9.3 第三层：描述子从 17.9% 轨迹覆盖扩大到“近全量相似森林”

当前 track delta 只在 verified match graph 内选父子，因此只有 224,119 / 1,251,246，即 17.9% 描述子得到预测。其余 944,506 个 unmatched descriptors 并不等于“彼此没有统计相似性”，只代表它们没有进入最终验证轨迹。这里存在当前数据库最大的结构机会。

新方法不依赖自研近邻算法：使用 Meta 的 MIT 许可 [Faiss](#) 仅在编码端为每个 `uint8[128]` 描述子寻找候选父节点。为保证 DAG，候选限制在编号更早、同项目、可选同图像邻域/相邻位姿窗口内；真正的边代价不是 L2，而是“父 ID 编码成本 + 128 维 modulo-256 residual 经快速试压后的 bit 数”。每个节点也保留 literal 候选，只有 parent + residual 严格更小时才选父。

这仍然是严格无损：存的是父描述子的完整字节加 128 个精确残差，解码逐维模 256 相加可恢复原 `uint8`。Faiss 的近似搜索即使找错邻居，也只会让 residual 不够小，不会改任何数据。Faiss 索引不进入归档，也不进入 iPhone 解码路径；编码可以在后台或云端运行很久。相比继续调 ZPAQ level，这条路线有机会把预测覆盖从 17.9% 推向绝大多数描述子，因而是数据库方向的第一优先级。

Meta 2025 年公布了专门的 vector ID 无损压缩研究，使用 Elias-Fano、wavelet tree、Random Edge Coding 等压缩 ANN 图/倒排表 ID；其官方代码能复现实验，但采用 CC BY-NC 4.0，不能直接用于商业产品。[Meta vector ID compression](#) 因此它只能证明“父 ID、match ID、邻接列表值得做专用图编码”，生产应使用 Apache-2.0 的 WebGraph

Rust 路线、Elias-Fano 基础结构或洁净实现。WebGraph 官方采用 gap compression、reference lists、intervalization 与 ζ codes，还支持压缩图上的延迟访问。[WebGraph](#)

9.4 第四层：结构分流之后再用真正的训练后端

描述子森林输出至少拆成：roots、parent IDs、每个维度的 residual lanes、literal lanes、逆置换；matches 拆成图像对、邻接列表、row IDs、inlier flags；关键点/位姿按 IEEE bit pattern 分列。然后对同一个冻结结构输出做后端竞争：

1. ZPAQ 7.15 method 5，作为成熟基线；
2. Pcodec v1.0.2 level 12，当前数值成员局部赢家；
3. OpenZL parser + clustering + ACE 完整训练，而非旧 brute-force selector；
4. C-Blosc2 B2ND 的 BYTEDELTA/SHUFFLE/BITSHUFFLE、always/never split、chunk/block 搜索，再接 Zstd；
5. 小 ID/offset 流使用 WebGraph/Elias-Fano/bitpack；
6. 每块解码验证后，只保留最小候选。

这仍然不是“自己发明熵模型”。PocketWorld 只定义语义流、父子关系和无损契约；最终熵编码使用成熟库。OpenZL 的价值正是把 parser、可逆 graph 与 ACE 训练组合起来；Blosc2 的价值是 B2ND 双层分块和可逆 filters；Pcodec 的价值是数值 sequence/page。这些库不再被当成互斥的整库冠军。

9.5 照片必须走真正的集合编码，而不是旧顺序启发式

照片仍是最大项。新的照片研究臂应完整包含：

- 用视觉特征、ARKit/SfM 位姿和快速 DCT 代价在整个作品中选择全局参考树，而非固定“上一张”；
- 对预测照片同时计算位姿投影的全局候选与局部 DCT block search，逐块选择边信息 + residual 总成本更低者；
- 直接编码量化 DCT 系数和重建原 JPEG 所需的 marker/table/restart/entropy side information，不解成 RGB 再重编码；
- GOP/参考深度限制为 4/8 张随机读取边界，根照片用当前 JXL/Brunsl/原 JPEG 最小者；
- 每张恢复后必须与源 JPEG 长度、逐字节和 SHA-256 完全一致。

微软 2016 集合方法是这个结构的主要公开证据；2024 年的“similar image JPEG frequency-domain block matching”进一步提出在量化 DCT 上搜索相似块、编码方向向量和 residual，但相关专利申请仍在，公开可商用实现也未找到，因此只能作为研究概念和专利审查对象，不能直接抄进产品。[2024 频域块匹配专利页面](#)

CVPR 2022 的 learned exact JPEG recompression 在 DCT 域用多层跨通道熵模型，论文报告胜过 Lepton、JPEG XL 和 CMIX；2023 年还有频域预测方向继续推进。它们说明单 JPEG 的统计模型仍有提升空间，但当前没有找到官方、成熟、许可清晰并可直接嵌入 iPhone 的实现，所以是中长期模型研究，不是马上上线的库。[CVPR 2022 原论文](#)

另一个可落地的大厂思想是 ROMP：用大量照片训练一组共享上下文 Huffman tables，把模型开销摊到整个语料库，原 JPEG 可 bit-wise 恢复。论文报告相对标准 JPEG 约节省 15%、相对 optimized JPEG 约 13%，不一定胜当前 JXL，但“跨大量作品共享训练模型而非每文件自带模型”值得和 WorldPack 结合。ROMP 官方仓库存在训练和 codec 源码，却没有发现明确 LICENSE 文件，因此商用状态是 **insufficient-evidence**，只能在隔离研究中评估思想。[ROMP 论文](#)、[ROMP 官方仓库](#)

9.6 源头写入：以后不再先制造一个不适合压缩的物理历史

对未来项目，authoritative source 应是 WorldPack 的逻辑对象，而不是 SQLite 页面历史。COLMAP/训练管线需要 SQLite 时，通过只读 view、虚拟表或短期物化缓存提供；需要 JPEG/PLY 时按块恢复。所有照片、描述子、匹配、关键点、位姿、稀疏/最终点云的逻辑数据都保留，没有删除模块。

这里“源头变小”不是丢数据，而是从第一天就不写无意义的 B-tree 页面排列、空闲页、重复导出和跨阶段副本。对新格式自身逐块 SHA；若业务需要重建传统文件，则保存足够的确定性导出版本/side information。旧项目继续使用文件级逐字节归档，不把新逻辑边界倒灌到历史数据。

十、接下来的实验顺序与现实倍率

10.1 不再同时改结构和后端

下一轮只做四个顺序清晰的大实验，不再散跑几十个小 codec：

顺序	冻结输入与 A/B	目的	停止/升级规则
1	A=track_delta_v1 + ZPAQ 124,401,918 B；B=全量 descriptor similarity forest + 相同 ZPAQ	单独测“扩大预测覆盖”值多少钱	B 严格更小即成为同作用域新基线；不小则改父节点/成本模型，不换后端掩盖问题
2	固定实验 1 的同一字节流；ZPAQ / 完整 ACE-trained OpenZL / B2ND+BYTEDELTA / Pcodec 竞争	补上以前没完整用的官方能力	每块选最小；所有 candidate 必须逆解逐字节一致
3	约 100 MB 连续 JPEG；A=当前 JXL exact；B=全局参考树 + 位姿/SfM + 局部 DCT block matching	正确重跑跨照片大结构	每张 JPEG SHA、随机 4/8 张、边信息全计入；比 A 小即升级研究基线
4	约 200 MB 完整作品；A=当前混合方案；B=WorldPack 内容寻址 + 相似排序 + 语义 delta	测完整作品净收益和跨文件重复	不以合成近重复数据冒充真实项目结果；报告各层独立贡献

主机实验仍只做可行性和快速否决，不批准生产赢家。通过的候选必须进入独立真机 bundle，在真实 iPhone/真实移动管线验证：全部严格无损、生产总时长回退不超过 3%、无超过 100 ms UI stall、生产管线开始时能够中断、后台能在块边界续跑、归档最后只保留一个已验证候选。

10.2 一致性门禁不变

- 原 JPEG 恢复后整文件 SHA-256、长度和逐字节一致；
- descriptor 每个 uint8[128]、数量、原 ordinal 与所有引用一致；
- keypoint/pose/camera/point cloud 每个浮点的 IEEE 位模式一致；
- match/two-view 的 ID、数量、方向、顺序契约一致；
- 随机 image/track/row/block 读取与原数据一致；
- 物化 SQLite 的所有逻辑 cell/row/order 一致且 integrity_check=ok；
- 稀疏点云、最终 PLY 和完整生产管线输出一致；
- 损坏块、错误版本、缺失父对象必须 fail closed；
- 原件只有在候选完整逆解验证、索引原子提交和审计状态落盘后删除。

10.3 5×/10×不是换一个库就能保证，但大结构能检验它是否存在

用当前代表性构成粗算：照片约 436 MB、SQLite 约 199 MB、PLY 约 8 MB，原总量约 643 MB。下表只是结构敏感性分析，不是结果预测：

假设场景	照片	DB	PLY/小文件	总量	对原约倍率
当前已知量级：照片按 JXL 历史 18.2% 节省，DB=track baseline	~356.6 MB	124.4 MB	~5-8 MB	~486-489 MB	~1.32×
照片达到论文平均 31% 节省，DB 仍 124.4 MB	~300.8 MB	124.4 MB	~5-8 MB	~430-433 MB	~1.49×
照片 31% 节省，DB 假设达到 2.5×	~300.8 MB	~79.6 MB	~5-8 MB	~385-388 MB	~1.66×-1.67×
照片假设减半，DB 2.5×	218 MB	~79.6 MB	~5-8 MB	~303-306 MB	~2.10×-2.12×
整体 5×目标	-	-	-	<=128.6 MB	5×
整体 10×目标	-	-	-	<=64.3 MB	10×

这说明照片单项决定上限：即使数据库做到 $2.5\times$ ，照片只减少 31% 时整体仍约 $1.67\times$ 。要达到 $5\times$ ，不是把 ZPAQ level 从 5 调到 22，而是必须在真实作品里发现非常强的跨照片/跨项目条件冗余。连续拍摄、真实位姿、同一场景和多阶段派生数据确实可能提供这种冗余；DwarFS 在大量版本数据上的巨大倍率证明结构上可能出现数量级收益，但不能外推到 PocketWorld。WorldPack 的价值就是把这些冗余分层测出来：exact duplicates 贡献多少、similarity delta 贡献多少、语义预测贡献多少、熵后端贡献多少。

如果真实作品实验最终仍只有 $1.5\times-2\times$ ，那不是“全世界没有更强压缩”，而是该真实输入在当前严格无损边界下没有足够可利用的重复。反之，如果相似森林和跨照片参考树让 residual 大量接近零，倍率会自然上升；不需要牺牲任何数据。当前证据支持继续向大结构投入，但不支持先承诺 $5\times/10\times$ 。

十一、开源商用初审

以下是工程初审，不是法律意见。**allow** 只在完整依赖/专利/NOTICE 审计完成后使用，本轮大多保持 **conditional**。

项目/固定身份	许可与风险	判定	原因
libjxl 稳定版	BSD-3-Clause 方向	conditional	已有精确 JPEG 能力；仍按实际 vendor revision 审计
OpenZL v0.2.0 / 3dceb648...	BSD 风格；Zstd/LZ4 permissive	conditional	新格式、无 iOS 证据、资源限制、需完整 transitive audit
Pcodec v1.0.2	Apache-2.0	conditional	Rust C ABI 不成熟，需固定 wrapper 与 Cargo 依赖
ALP / 冻结 revision	MIT	conditional	C++17/Clang；M1 ARM 证据不等于 iPhone
C-Blosc2 固定 release	BSD；可 bundled 多 codec	conditional	必须审计实际启用依赖并禁用有损 filters
Faiss 固定 release	MIT	conditional	仅用于离线/编码端找父节点；不把近似索引或量化结果作为数据真源
WebGraph Rust 固定 release	Apache-2.0 或 LGPL-2.1+ 双许可方向	conditional	只用于 match/parent ID 图流，需确认选择的 Rust crate 与实际依赖
DwarFS writer	GPL-3.0；reader/部分库 MIT	conditional/reference-only	可用于隔离主机/服务端 benchmark；不直接嵌入闭源 iPhone writer
OpenVCDiff	Apache-2.0，但仓库已归档	conditional/reference-only	可作 delta 诊断基线，不宜成为新永久格式核心依赖
Meta vector ID compression	CC BY-NC 4.0	block for commercial code	只借鉴公开论文思想，不能把官方代码用于商业产品
ROMP 官方仓库	未发现明确 LICENSE	insufficient-evidence	有训练/codec 源码，但许可不清，不能商业集成
Arrow/Parquet	Apache-2.0	conditional	许可友好但移动依赖和格式复杂度高
TileDB	permissive 方向、依赖较多	conditional	作为引擎过重，完整依赖审计未完成
shuffle-coding d3e92bc...	Cargo 声明 MIT，仓库未见 LICENSE	insufficient-evidence	不能仅凭 manifest 批准商业分发
TMC13/G-PCC a3d15...	BSD 风格但专利权未授予	block/conditional	专利与严格语义双重问题
Draco 472389...	Apache-2.0	conditional	许可较清晰，但原 PLY bitstream 精确性未获官方承诺
fzip/zfp exact	BSD 风格方向	conditional	需固定 revision、禁用有损模式并真机复验
2016 JPEG collection / LLJPEG 论文	无生产代码许可证	insufficient-evidence	只能洁净室复现/另行专利和许可分析

十二、证据台账与置信度

结论	证据	权威性	限制	状态
当前 DB 80.5% 是 descriptors	冻结 SQLite dbstat、SHA、实验 YAML	一手本地数据	单个代表项目，需五项目复验	confirmed
track delta 只覆盖 17.9% 描述子且省 2.455%	三轮确定性实验结果	一手本地数据	主机预筛，未真机	confirmed
exact v2 比 track 大 64,155 B	冻结结果 YAML	一手本地数据	单样本，但已触发预设否决	confirmed
格式感知"parse/group/transform/compress"是主方向	OpenZL 官方论文/文档、Parquet/TileDB/Blosc 2 规范	一手官方/同行评审	不保证 PocketWorld 比率	supported
跨 JPEG collection 可比单文件更小	IEEE TIP/VCIP 原论文	同行评审原文	无成熟 OSS，数据集不同	supported
公开照片方案没有证明单项目 2.165×	已查论文报告区间与实现状态	系统检索后的否定结论	旧门槛已取消；也无法证明世界上不存在私有方案	supported
旧 OpenZL 测试没有跑 ACE training	bridge 源码、6-member 结果、OpenZL 官方流程	本地源码 + 官方文档	完整训练结果尚未测试	confirmed
旧 Blosc2 测试没有 BYTEDELTA/B2ND	bridge 白名单、6-member 结果、Blosc2 官方文档	本地源码 + 官方文档	完整组合结果尚未测试	confirmed
旧跨照片原型不是 2016 论文忠实复现	原型源码、12.85% 映射覆盖、论文方法	本地源码 + 同行评审原文	忠实结构 A/B 尚未实施	confirmed
Pcodec 将 PWA2 同作用域基线改善 366,443 B	66-member 完整结果与 baseline-decision	一手本地数据	不是全 DB/生产基线	confirmed
SIFT 5×--16× headline 多为无损/任务无损	SIFT 压缩综述与原论文	同行评审/综述	个别严格集合编码除外	confirmed
G-PCC 10×不能代表原 PLY 文件精确恢复	MPEG/TMC13 文档与编码前整数化语义	官方/参考软件	规范化整数点云可另行采用	confirmed
OpenZL 可直接上 iPhone 生产	无官方 iOS 证据，社区接口曾有问题	证据不足	必须实测和审计	unresolved
全量 descriptor similarity forest 能胜 124,401,918 B	尚未实施	假设	这是下一实验，不是承诺	not checked
严格无损整体能达到 5×--10×	当前无公开或本地证据	证据不足	受照片条件熵支配	unresolved

最终决策

审计后的决策发生了变化。以前不应把 OpenZL、Blosc2、跨照片集合编码写成失败：它们没有被完整使用；Brunsli、Lepton、PackJPG、DwarFS、Parquet/ORC/TileDB 等也没有耐久结果。真正成立的局部事实只有 JXL、ZPAQ、track delta、exact v2、PWA2 和 Pcodec 这些有哈希/结果文件支撑的实验。

下一步不做 `exact_transform_v3`，也不再散跑更多通用压缩器。第一优先级是全量 descriptor similarity forest + 相同 ZPAQ，把预测覆盖从 17.9% 扩向几乎所有描述子；第二步才在完全相同的结构字节流上补跑官方 ACE-trained OpenZL、B2ND+BYTEDelta 和 Pcodec。照片方向单独做忠实的全局参考树 + 位姿/SfM + 局部 DCT block matching，与当前 JXL exact 基线 A/B。完整作品最终进入 WorldPack：内容寻址去重、相似对象参考树、语义流、成熟后端逐块选小。

同作用域中任何严格更小、严格无损的候选都升级研究基线，不再设 10% 人工门槛；是否侵入生产则按完整作品净收益、解码边界、真机资源、后台事务和许可证共同判断。这条路线不删除照片、特征、匹配、关键点、位姿或任何点云，不量化、不舍入，也不依赖自研熵编码器。PocketWorld 要创新的是怎样暴露跨文件、跨照片、跨轨迹的条件冗余，成熟库负责最后的可逆编码。

主要来源

- [OpenZL 官方介绍、结构化压缩文档](#)
- [Apache Parquet 编码规范、Apache Arrow Columnar Format](#)
- [C-Blosc2/B2ND 官方文档](#)
- [TileDB 数据布局与 tile 文档](#)
- [Pcodec 官方仓库、Pcodec 论文](#)
- [ALP 官方仓库、ALP 论文 DOI](#)
- [Lossless Compression of JPEG Coded Photo Collections](#)
- [Incremental SfM based lossless compression of JPEG coded photo album](#)
- [Practical Learned Lossless JPEG Recompression](#)
- [JPEG XL 官方站点、libjxl](#)
- [Survey of SIFT Compression Schemes](#)
- [Compressing Sets and Multisets of Sequences、Practical Shuffle Coding](#)
- [SQLite 文件格式、VACUUM、WITHOUT ROWID](#)
- [zfp reversible mode、fpzip](#)
- [Google Draco、MPEG G-PCC 资料](#)
- [MCAP 规范、restic 内容定义分块设计](#)
- [Git Packfiles、DwarFS、OpenVCDiff](#)
- [Faiss、WebGraph、Meta vector ID compression](#)
- [C-Blosc2 BYTEDELTA 官方实验、OpenZL 正确使用流程](#)
- [ROMP 大规模共享 JPEG 上下文模型](#)